

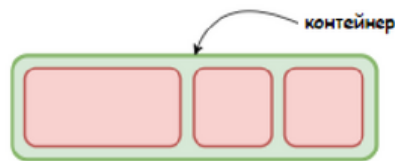
CSS, Flex-box

Flex-box

Модуль Flexbox Layout ([W3C Candidate Recommendation](#)) используется для более эффективного способа изменения размера, размещения, выравнивания и распределения пространства между элементами в контейнере.

Основная идея Flexbox состоит в том, что контейнер имеет возможность изменять размер, порядок его вложенных элементов, чтобы наилучшим образом заполнить доступное пространство. Это используется при создании страниц для отображения на всех типах устройств с любым размером экрана. Flex контейнер, например, расширяет элементы, чтобы заполнить доступное свободное пространство, или сжимает их, чтобы предотвратить переполнение.

Свойства flex-контейнера



display

Для создания flex-контейнера используется свойство **display**:

```
.container {  
  display: flex | inline-flex;  
}
```

Значения свойства:

- **flex** - контейнер, который имеет свойства блочного элемента;
- **inline-flex** - контейнер, который имеет свойства строчного элемента.

Свойства flex-контейнера

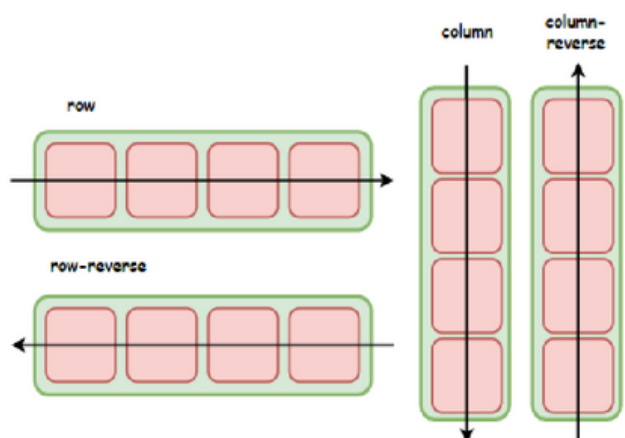
flex-direction

Устанавливает ось, которая определяет направление вложенных в контейнер элементов.

```
.container {  
  flex-direction: row | row-reverse | column | column-reverse;  
}
```

Значения свойства **flex-direction**:

- **row** (по умолчанию): слева направо;
- **row-reverse**: справа налево;
- **column**: сверху вниз;
- **column-reverse**: снизу вверх.



Flex-контейнер, пример

Пример. Создадим flex-контейнер и разместим в него несколько блоков, установив для них направление размещения справа налево и отображение **flex**.

Код CSS

```
.container {  
  display: flex;  
  flex-direction: row-reverse;  
  background: LightGreen;  
}  
.block {  
  min-width: 50px;  
  height: 50px;  
  margin: 2px;  
  border: solid thin green;  
}
```

Результат



Flex-контейнер занимает все ширину окна браузера, как и элемент с блочным отображением.

Flex-контейнер, пример

Пример. Создадим flex-контейнер и разместим в него несколько блоков, установив для них направление размещения справа налево и отображение **inline-flex**.

Код CSS

```
.container {  
  display: inline-flex;  
  flex-direction: row-reverse;  
  background: LightGreen;  
}  
.block {  
  min-width: 50px;  
  height: 50px;  
  margin: 2px;  
  border: solid thin green;  
}
```

Результат



Контейнер ведет себя как **inline** элемент - подстраивает ширину под содержимого

Свойство flex-wrap

По умолчанию вложенные элементы в flex-контейнере будут пытаться расположиться на одной строке. Чтобы изменить это поведение, и "позволить" элементам переходить на новую строку по мере необходимости, используется свойство **flex-wrap**:

```
.container {  
  flex-wrap: nowrap | wrap | wrap-reverse;  
}
```

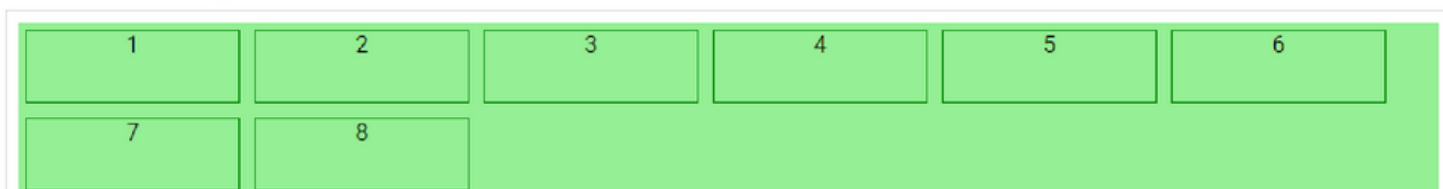
Свойство flex-wrap

Значения свойства **flex-wrap** :

- **nowrap** (по умолчанию): все вложенные элементы в одной строке, при этом они "сжимаются" по ширине, чтобы поместиться в контейнере:



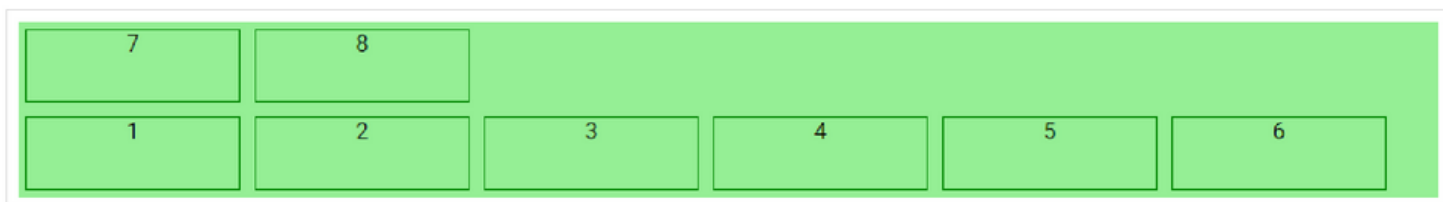
- **wrap**: перенос элементов, ширина вложенных блоков остается как указано в стиле CSS, а те блоки, которые не помещаются, переносятся на новую строку, при этом сначала заполняется верхняя строка, а затем уже нижняя:



Свойство flex-wrap

Значения свойства **flex-wrap** :

- **wrap-reverse**: вложенные элементы будут перенесены на несколько строк , ширина вложенных блоков остается как указано в стиле CSS, а те блоки, которые не помещаются, переносятся на новую строку, при этом сначала заполняется нижняя строка, а затем уже верхняя:



Задание

Задание 1. Дан HTML-код и CSS-код. Какой результат получится на странице?

```
<div class="container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
  <div>7</div>
  <div>8</div>
</div>
```

```
.container {
  display: flex;
  flex-wrap: no-wrap;
  background: LightGreen;
}
.container div {
  min-width: 120px;
  height: 50px;
  margin: 5px;
  border: solid thin green;
}
```

Свойство justify-content

Это свойство определяет выравнивание элементов вдоль оси flex-контейнера:

```
.container {  
    justify-content: flex-start | flex-end |  
                    center | space-between |  
                    space-around | space-evenly  
}
```

Свойство justify-content

Значения свойства (для **flex-direction: row**):

- **flex-start** (по умолчанию) - элементы сдвинуты в начало **flex-direction** направления:



- **flex-end** - элементы сдвинуты к концу **flex-direction** направления:



- **center** - элементы выровнены по центру контейнера:



Свойство justify-content

Значения свойства (для **flex-direction: row**):

- **space-between** - элементы равномерно распределены вдоль flex-направления: первый элемент находится в начале строки, последний элемент в конце строки:



- **space-around** - элементы равномерно распределены вдоль flex-направления с одинаковым пространством вокруг них:



Свойство justify-content

Значения свойства (для **flex-direction: row**):

- **space-evenly** - элементы распределяются таким образом, чтобы расстояние между любыми двумя элементами (и расстояние до границ) было одинаковым:



Свойство align-items

Это свойство определяет выравнивание вдоль поперечной оси, расположенной перпендикулярно оси, установленной (главной) в flex-контейнере:

- главная ось: **row** (слева направо) - поперечная ось: **column** (сверху вниз);
- главная ось: **row-reverse** (справа налево) - поперечная ось: **column-reverse** (снизу вверх);
- главная ось: **column** (сверху вниз) - поперечная ось: **row** (слева направо);
- главная ось: **column-reverse** (снизу вверх) - поперечная ось: **row-reverse** (справа налево).

Свойство align-items

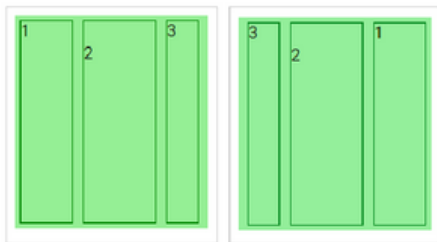
Это свойство определяет выравнивание вдоль поперечной оси, расположенной перпендикулярно оси, установленной (главной) в flex-контейнере:

```
.container {  
  align-items: stretch | flex-start | flex-end | center | baseline  
}
```

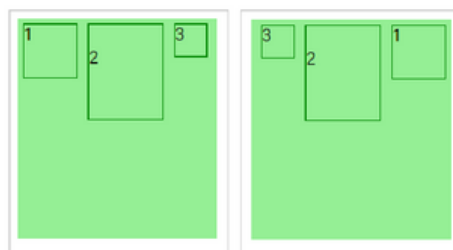
Свойство align-items

Значения свойства (для **flex-direction: row** и **row-reverse**)

- **stretch** (по умолчанию) - элементы "растягиваются" вдоль поперечной оси, чтобы заполнить контейнер:



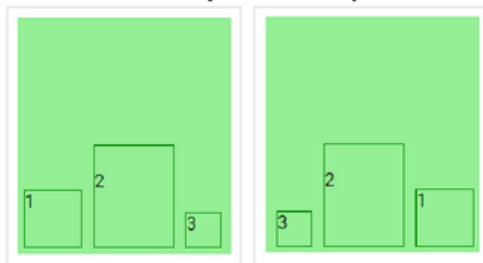
- **flex-start** - элементы размещаются в начале поперечной оси:



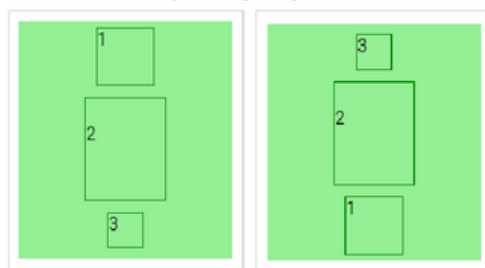
Свойство align-items

Значения свойства (для **flex-direction: row** и **row-reverse**)

- **flex-end** - элементы размещаются в конце поперечной оси:



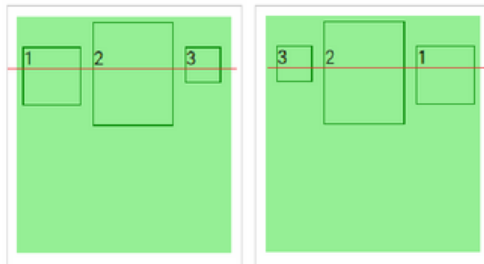
- **center** - элементы центрированы по поперечной оси:



Свойство align-items

Значения свойства (для **flex-direction: row** и **row-reverse**)

- **baseline** - элементы выровнены по линии текста:



Свойство gap

gap — это расстояние между отдельными элементами, строками или колонками внутри flex-контейнера.

Значение по умолчанию — **none**.

Задаётся пикселями или процентами.

Свойство gap

Существует три версии свойства **gap**:

- **row-gap** — устанавливает расстояние между строками;
- **column-gap** — устанавливает расстояние между колонками;
- **gap** — устанавливает расстояние между строками и колонками.

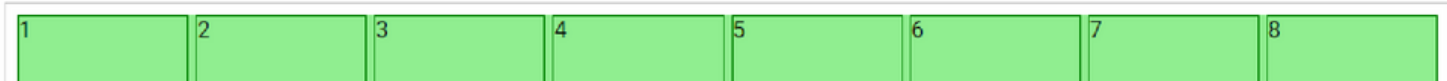
С помощью **gap** расстояния устанавливаются только между элементами, а не между элементами и границами родителя.

Свойство gap

Пример.

```
<div class="container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
  <div>7</div>
  <div>8</div>
</div>
```

```
.container {
  display: flex;
  flex-wrap: no-wrap;
  background: LightGreen;
  gap: 5px;
}
.container div {
  width: 120px;
  height: 50px;
  border: solid thin green;
}
```



Задание

Задание 2. Дан HTML-код:

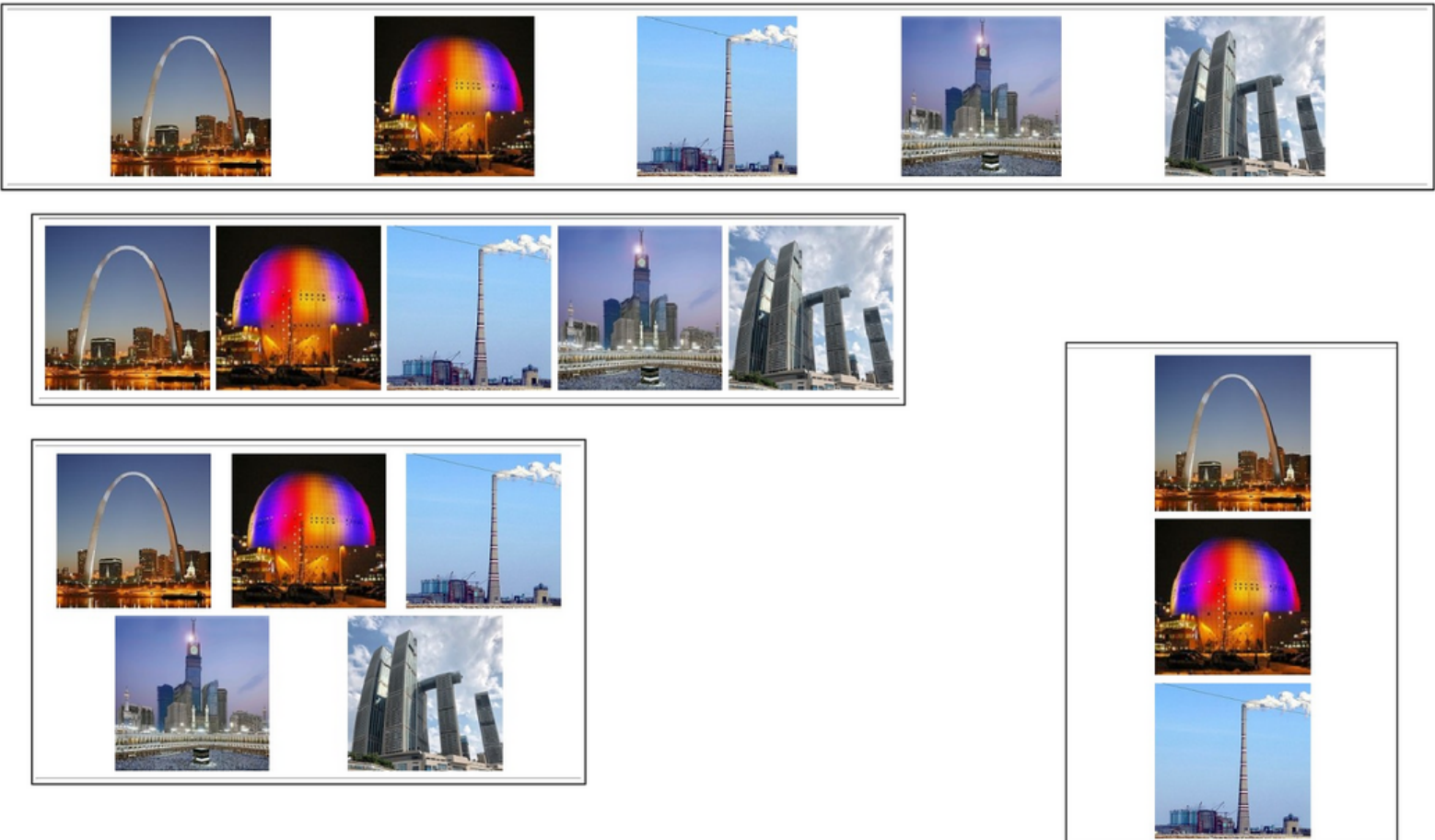
```
<hr>
<div class="pict">
  <a href="#"></a>
  <a href="#"></a>
  <a href="#"></a>
  <a href="#"></a>
  <a href="#"></a>
</div>
<hr>
```

Описать стили для этого фрагмента так, чтобы рисунки оригинального размера располагались в одну строку с одинаковыми интервалами между ними.

При уменьшении экрана до 1000 пикселей, рисунки должны перестраиваться, сохраняя исходный размер и способ размещения в блоке.

При уменьшении экрана до 700 пикселей, рисунки должны перестроится в одну колонку и разместиться по центру.

Задание



Свойства flex-элементов

Flex-элементы - это дочерние элементы flex-контейнера, то есть вложенные непосредственно в него элементы.

Способы их отображения определяются в родительском flex-контейнере, при этом они одинаковы для всех элементов.

Но модель flexbox позволяет задавать свойства отдельным flex-элементам.

Свойства flex-элементов

- **order** - позволяет изменить порядок следования flex-элементов.

Свойство может иметь значение, большее или равное нулю (0 - установлен по умолчанию – сохраняется исходный порядок следования элементов).

Если значение больше 0, то элементы при отображении «сортируются» по этому значению.

Свойства flex-элементов

- **flex-grow** - способность flex-элемента растягиваться в случае необходимости.

Свойство может иметь значение, большее или равное нулю (0 - установлен по умолчанию – растяжения нет).

Это свойство указывает, какое количество доступного пространства внутри flex-контейнера должен занимать элемент.

Если для всех элементов **flex-grow** установлено значение 1, оставшееся пространство в контейнере будет равномерно распределено между всеми flex-элементами.

Если один из элементов имеет значение **flex-grow** 2, этот элемент займет в два раза больше места, чем остальные (если это возможно).

Свойства flex-элементов

- **flex-shrink** - способность flex-элемента сжиматься в случае необходимости.

Свойство может иметь значение, большее или равное нулю, которое определяет пропорцию.

Это свойство применяется если суммарная ширина/высота flex-элементов превышает ширину/высоту flex-контернера и не установлен перенос на следующую строку.

Свойства flex-элементов

- **flex-basis** - определяет размер элемента по умолчанию перед распределением свободного пространства. Это может быть длина (например, 20%, 5em и т.д.) или **auto**.

Свойства flex-элементов

- **flex** - это сокращение для использования **flex-grow**, **flex-shrink** и **flex-basis** вместе.

Второй и третий параметры (**flex-shrink** и **flex-basis**) являются необязательными.

По умолчанию установлено **0 1 auto**.

Свойства flex-элементов

- **align-self** - позволяет переопределить выравнивание по умолчанию (или указанное с помощью **align-items**) для отдельных flex-элементов.

Возможные значения (соответствуют значениям **align-items**):

```
flex-элемент {  
  align-self: auto | flex-start | flex-end | center | baseline | stretch;  
}
```

Задание

Задание 3. Дан HTML-код:

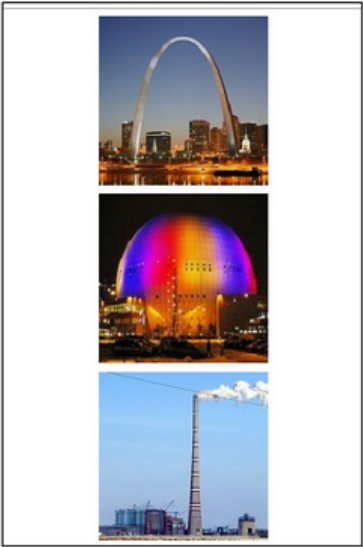
```
<hr>  
<div class="pict">  
  <a href="#"></a>  
  <a href="#"></a>  
  <a href="#" class="big"></a>  
  <a href="#"></a>  
  <a href="#"></a>  
</div>  
<hr>
```

Описать стили для этого фрагмента так, чтобы рисунки оригинального размера располагались в одну строку с одинаковыми интервалами между ними. Рисунок класса **big** должен быть больше остальных на 20%

При уменьшении экрана до 1000 пикселей, рисунки должны перестраиваться. Рисунок класса **big** должен стать как остальные рисунки.

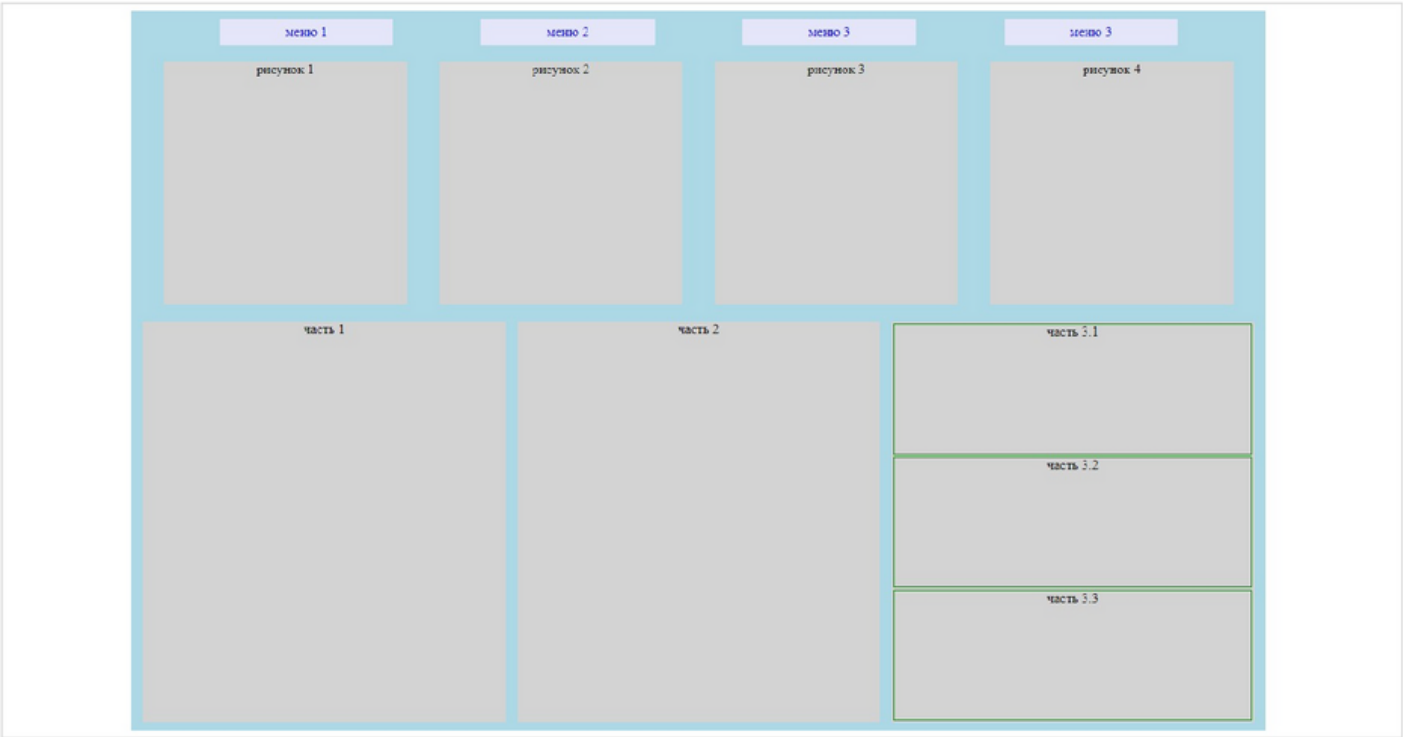
При уменьшении экрана до 700 пикселей, рисунки должны перестроиться в одну колонку и разместиться по центру.

Задание



Flexbox, пример верстки

Пример. С помощью **flexbox** создать страницу следующей структуры:



Flexbox, пример верстки

Шаг 1. Реализуем HTML-код страницы

```
<div class="content">
  <a href="#">меню 1</a>
  <a href="#">меню 2</a>
  <a href="#">меню 3</a>
  <a href="#">меню 3</a>
</div>
<div class="content">
  <div class="image">рисунок 1</div>
  <div class="image">рисунок 2</div>
  <div class="image">рисунок 3</div>
  <div class="image">рисунок 4</div>
</div>
<div class="content">
  <div class="article">часть 1</div>
  <div class="article">часть 2</div>
  <div class="article">
    <div class="item">часть 3.1</div>
    <div class="item">часть 3.2</div>
    <div class="item">часть 3.3</div>
  </div>
</div>
```

Flexbox, пример верстки

Шаг 2. Описываем стили для каждого класса (без перестроения).

```
.content {
  width: 70%;
  display: flex;
  margin: 0 auto;
  padding: 0.5vmax 0;
  justify-content: space-evenly;
  text-align: center;
  background: lightblue;
}

.article {
  width: 32%;
  background: lightgrey;
}

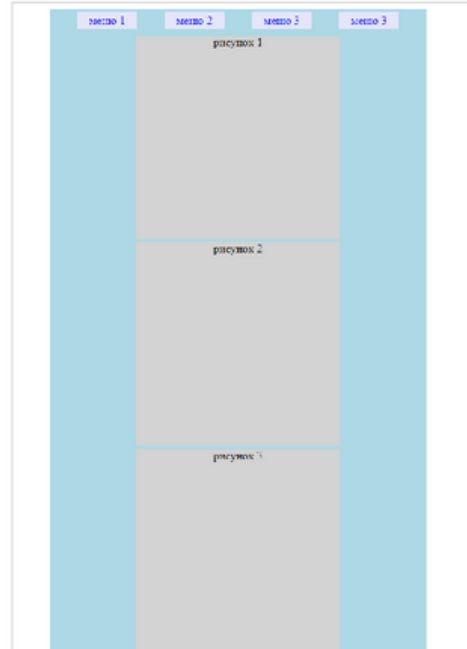
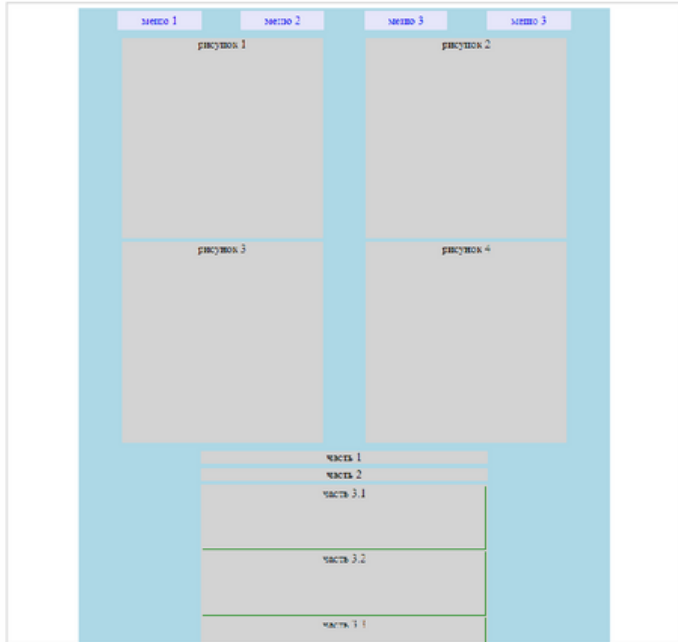
.image { /* квадратные */
  width: 15vmax;
  height: 15vmax;
  background: lightgrey;
}

.item {
  margin: 0.5%;
  height: 8vmax;
  border: thin solid green;
}

a {
  width: 10vw;
  padding: 0.5vw;
  text-decoration: none;
  background: Lavender;
}
```

Flexbox, пример верстки

При уменьшении ширины окна браузера элементы должны перестраиваться (в элементах часть 1 и часть 2 нет содержимого):



Flexbox, пример верстки

Шаг 3. Изменяем классы, чтобы элементы перестраивались

```
.content {  
  width: 70%;  
  display: flex;  
  margin: 0 auto;  
  padding: 0.5vmax 0;  
  justify-content: space-evenly;  
  text-align: center;  
  background: lightblue;  
  row-gap: 0.5vmax;  
  flex-wrap: wrap;  
}  
.article {  
  width: 32%;  
  background: lightgrey;  
  min-width: 400px;  
}
```

```
.image { /* квадратные */  
  width: 15vmax;  
  height: 15vmax;  
  background: lightgrey;  
  min-width: 280px;  
  min-height: 280px;  
}  
.item {  
  margin: 0.5%;  
  height: 8vmax;  
  border: thin solid green;  
}  
a {  
  width: 10vw;  
  padding: 0.5vw;  
  text-decoration: none;  
  background: Lavender;  
}
```

Спасибо за внимание!